

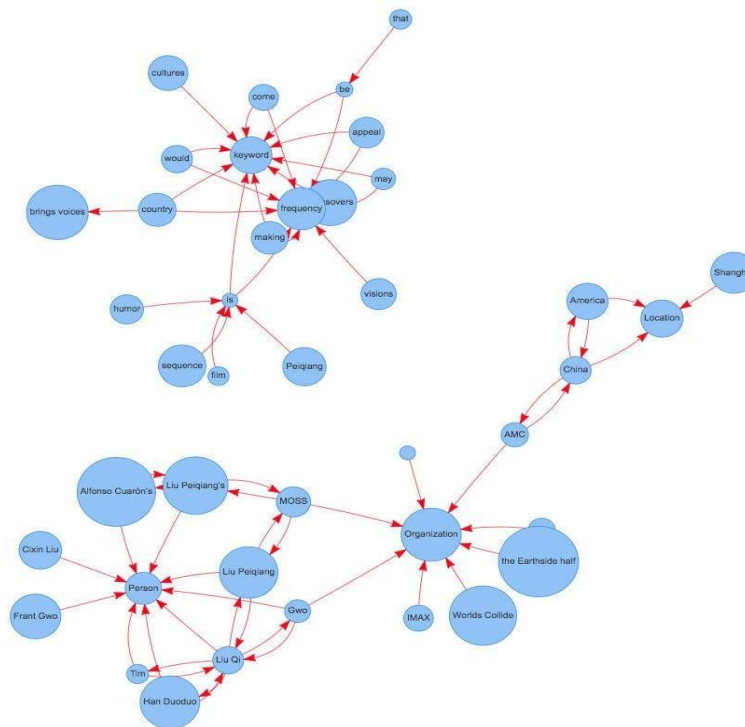
 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

Aim: TextRank for Document Summarization

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.



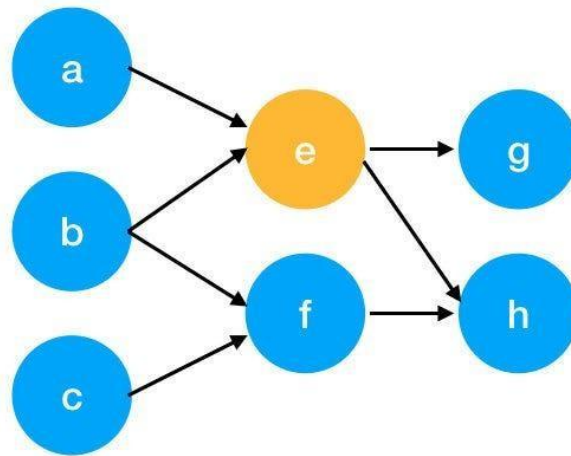
Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

$$S(V_i) = (1 - d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- $S(V_i)$ - the weight of webpage i
- d - damping factor, in case of no outgoing links
- $In(V_i)$ - inbound links of i, which is a set
- $Out(V_j)$ - outgoing links of j, which is a set
- $|Out(V_j)|$ - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$\begin{aligned}
 In(v_e) &= \{a, b\}, j \in \{a, b\} \\
 \sum_{j \in \{a, b\}} \frac{1}{|Out(V_j)|} S(V_j) &= \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b) \\
 &= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b) \\
 &= S(V_a) + \frac{1}{2} S(V_b)
 \end{aligned}$$

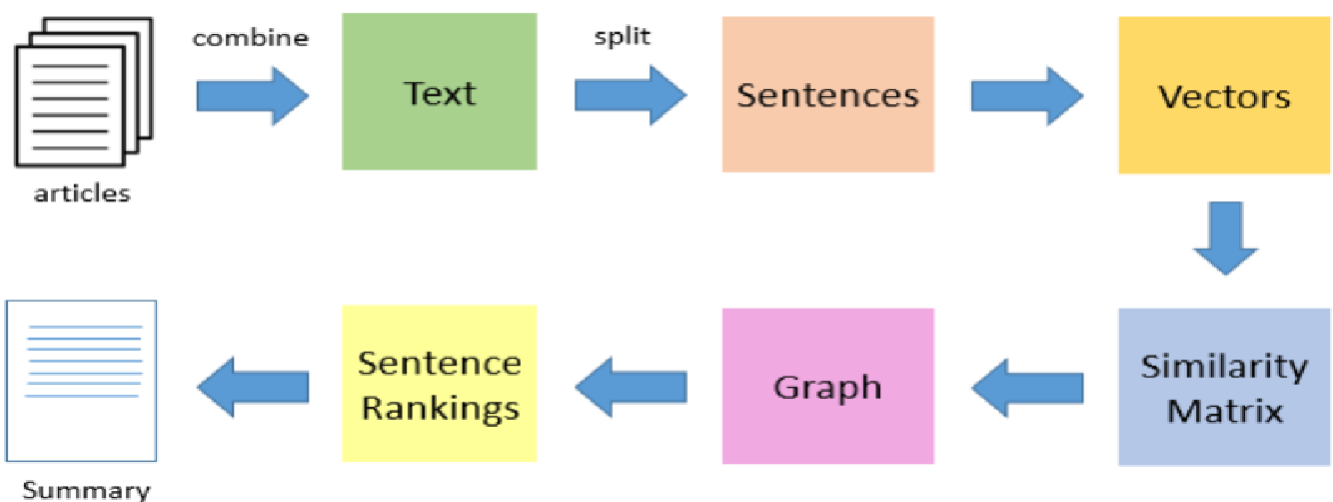
We can get the weight of webpage e by the following function.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

$$S(V_e) = (1 - d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

TextRank for document Summarization



TextRank works in the following steps:

1. Tokenize documents into sentences.
2. Preprocess each sentence in the document.
3. Count key phrases and normalize them or produce TFIDF Matrix, you can also use any kind of vectorization such as spacy vectors.
4. Calculate the Jaccard Similarity between sentences and key phrases.
5. Rank the sentences with higher significance.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

Pre Lab Exercise:

1. How keyword extraction is similar to document summarization process?
they are similar because both aim to condense large text into its most important information. While keyword extraction highlights crucial terms, summarization captures the main ideas, making both processes about reducing text to its essence.
2. How keyword extraction is different to document summarization process?
Keyword extraction focuses on selecting specific important words or phrases from text. Document summarization, on the other hand, generates coherent sentences capturing the overall meaning and context.
3. Limitation of TextRank as a summarizer
TextRank as a summarizer is limited because it is purely extractive, depends heavily on word co-occurrence graphs, and often struggles with context, coherence, and abstractive meaning. It can miss semantic nuances, produce redundant sentences, and is not ideal for highly technical or creative texts.
 - Extractive only: It selects sentences directly from the text, without generating new ones, so summaries may lack abstraction.
 - Context insensitivity: Relies on word/sentence co-occurrence graphs, which can miss deeper semantic meaning.
 - Redundancy: Often includes overlapping or repetitive sentences due to weak semantic filtering.
 - Coherence issues: Summaries may feel disjointed or lack logical flow.
 - Domain limitations: Performs poorly on highly technical, structured, or creative texts.
 - No learning ability: Unlike modern neural models, TextRank doesn't adapt or improve with data.

Program (Code):

```
[1]
import nltk
import numpy as np
import networkx as nx
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from sklearn.metrics.pairwise import cosine_similarity
from collections import Counter

# Download necessary nltk resources
nltk.download("punkt")
nltk.download("stopwords")

def preprocess_text(text):
    stop_words = set(stopwords.words("english"))
    sentences = sent_tokenize(text)
    word_frequencies = []
    for sent in sentences:
        words = word_tokenize(sent.lower())
        filtered_words = [word for word in words if word.isalnum() and word not in stop_words]
        word_frequencies.append(Counter(filtered_words))

    return sentences, word_frequencies
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

```
def build_similarity_matrix(word_frequencies):
    size = len(word_frequencies)
    similarity_matrix = np.zeros((size, size))
    for i in range(size):
        for j in range(size):
            if i != j:
                words1 = word_frequencies[i]
                words2 = word_frequencies[j]
                common_words = set(words1.keys()).union(set(words2.keys()))
                vec1 = np.array([words1[word] for word in common_words])
                vec2 = np.array([words2[word] for word in common_words])

                similarity_matrix[i][j] = cosine_similarity([vec1], [vec2])[0, 0]

    return similarity_matrix

def textrank_summarization(text, num_sentences=3):
    sentences, word_frequencies = preprocess_text(text)
    similarity_matrix = build_similarity_matrix(word_frequencies)

    # Build graph and rank sentences using PageRank
    graph = nx.from_numpy_array(similarity_matrix)
    scores = nx.pagerank(graph)

    # Sort sentences by score and select top-ranked ones
    ranked_sentences = sorted(((scores[i], s) for i, s in enumerate(sentences)), reverse=True)
    summary = " ".join([sent for _, sent in ranked_sentences[:num_sentences]])

    return summary
```

Results:

```
[1]
✓ 7s
import nltk
import numpy as np
import networkx as nx
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from sklearn.metrics.pairwise import cosine_similarity
from collections import Counter

[2]
✓ 1s
# Download necessary nltk resources
nltk.download("punkt")
nltk.download("stopwords")

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True

[3]
✓ 0s
def preprocess_text(text):
    stop_words = set(stopwords.words("english"))
    sentences = sent_tokenize(text)
    word_frequencies = {}

    for sent in sentences:
        words = word_tokenize(sent.lower())
        filtered_words = [word for word in words if word.isalnum() and word not in stop_words]
        word_frequencies.append(Counter(filtered_words))

    return sentences, word_frequencies
```



Marwadi
University

Marwadi University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Artificial
Intelligence (01CT0616)**

Aim: TextRank for Document Summarization

Experiment No: 10

Date: 28/03/2026

Enrolment No: 92301733059

[4]
✓ Os

```
def build_similarity_matrix(word_frequencies):  
    size = len(word_frequencies)  
    similarity_matrix = np.zeros((size, size))  
  
    for i in range(size):  
        for j in range(size):  
            if i != j:  
                words1 = word_frequencies[i]  
                words2 = word_frequencies[j]  
                common_words = set(words1.keys()).union(set(words2.keys()))  
  
                vec1 = np.array([words1[word] for word in common_words])  
                vec2 = np.array([words2[word] for word in common_words])  
  
                similarity_matrix[i][j] = cosine_similarity([vec1], [vec2])[0, 0]  
  
    return similarity_matrix
```

[5]
✓ Os

```
def textrank_summarization(text, num_sentences=3):  
    sentences, word_frequencies = preprocess_text(text)  
    similarity_matrix = build_similarity_matrix(word_frequencies)  
  
    # Build graph and rank sentences using PageRank  
    graph = nx.from_numpy_array(similarity_matrix)  
    scores = nx.pagerank(graph)  
  
    # Sort sentences by score and select top-ranked ones  
    ranked_sentences = sorted(((scores[i], s) for i, s in enumerate(sentences)), reverse=True)  
    summary = " ".join([sent for _, sent in ranked_sentences[:num_sentences]])  
  
    return summary
```

[6]
✓ Os

```
nltk.download('punkt_tab')  
text = """TextRank is an unsupervised algorithm for keyword extraction and text summarization.  
It is based on PageRank, which is used by Google to rank web pages in search results.  
TextRank builds a graph of sentences, where edges represent similarity between them.  
By running the PageRank algorithm on this graph, we can extract the most important sentences  
for summarization. This technique is widely used in NLP applications."""
```

✓

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...  
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
```

[7]
✓ Os

```
summary = textrank_summarization(text, num_sentences=2)  
print("Summary:")  
print(summary)
```

✓

```
*** Summary:  
By running the PageRank algorithm on this graph, we can extract the most important sentences  
for summarization. TextRank is an unsupervised algorithm for keyword extraction and text summarization.
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

Observation:

- **Graph-Based Ranking:** It uses the **PageRank** algorithm (via `networkx`) to treat sentences as nodes in a graph, where the "importance" of a sentence is determined by its connections to others.
- **Similarity Measurement:** Connections between sentences are calculated using **Cosine Similarity**, which measures how many relevant words two sentences share in a vector space.
- **Text Preprocessing:** The script cleans the input by tokenizing sentences, removing **stop words** (like "the" or "is"), and converting text to lowercase to ensure mathematical comparisons are accurate.
- **Extractive Nature:** Unlike AI that rewrites text, this script is **extractive**, meaning it purely selects and pulls the highest-scoring original sentences to form a summary.
- **Unsupervised Efficiency:** The approach is **unsupervised**, allowing it to summarize any text immediately without requiring a labeled dataset or intensive machine learning training.

Post Lab Exercise:

1. Write about "your ownself" (in not more than 500 words → You know better about you, rather than ChatGPT!!). Generate the summary of your portfolio in
 - a. 50% of the size of your portfolio
 - b. 25% of the size of your portfolio

Comment over the summary obtained. Which sentence you think should be there in your summary, but was not spotted by textrank? Rate the overall summary in the scale of 1-5 (with 1 as least and 5 as highest rating). Paste the code, your portfolio, output and your analysis.

```

import nltk
import numpy as np
import networkx as nx
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from sklearn.metrics.pairwise import cosine_similarity
from collections import Counter

# Download necessary nltk resources
nltk.download("punkt")
nltk.download("stopwords")

def preprocess_text(text):
    stop_words = set(stopwords.words("english"))
    sentences = sent_tokenize(text)
    word_frequencies = []
    for sent in sentences:
        words = word_tokenize(sent.lower())
        filtered_words = [word for word in words if word.isalnum() and word not in stop_words]
        word_frequencies.append(Counter(filtered_words))

    return sentences, word_frequencies

def build_similarity_matrix(word_frequencies):
    size = len(word_frequencies)
    similarity_matrix = np.zeros((size, size))
    for i in range(size):
        for j in range(size):
            if i != j:
                words1 = word_frequencies[i]
                words2 = word_frequencies[j]

```


 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

```

        continue

    vec1 = np.array([words1.get(word, 0) for word in common_words])
    vec2 = np.array([words2.get(word, 0) for word in common_words])

    similarity_matrix[i][j] = cosine_similarity([vec1], [vec2])[0, 0]

return similarity_matrix

def textrank_summarization(text, num_sentences=3):
    sentences, word_frequencies = preprocess_text(text)

    # If the text is too short for the requested number of sentences
    if len(sentences) <= num_sentences:
        return " ".join(sentences)

    similarity_matrix = build_similarity_matrix(word_frequencies)

    # Build graph and rank sentences using PageRank
    graph = nx.from_numpy_array(similarity_matrix)
    scores = nx.pagerank(graph)

    # Sort sentences by score and select top-ranked ones
    ranked_sentences = sorted(((scores[i], s) for i, s in enumerate(sentences)), reverse=True)

    # Selecting the top 'num_sentences'
    summary = " ".join([sent for _, sent in ranked_sentences[:num_sentences]])

    return summary

```

```

*** [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

```

# 1. First, tokenize to get the total count
all_sentences = sent_tokenize(text)
total_count = len(all_sentences)

```

```

# 2. Calculate 50% of the size
# We use max(1, ...) to ensure at least one sentence remains
target_count = max(1, int(total_count * 0.5))

```

```

# 3. Generate and print the summary
summary_50_percent = textrank_summarization(text, num_sentences=target_count)

print(f"Original sentences: {total_count}")
print(f"Summary sentences (50%): {target_count}")
print("\n--- Summary ---")
print(summary_50_percent)

```

```

*** Original sentences: 17
Summary sentences (50%): 8

```

```

--- Summary ---
My professional focus lies at the intersection of Cloud Infrastructure, Mobile Development, and Artificial Intelligence, where I build scalable, high-performance solutions that bridge the gap between complex backend systems and intuitive user experiences. Core Expertise My technical foundation is built on a deep understanding of Cloud Engineering, specifically within the AWS (Amazon Web Services) ecosystem. Technical Toolkit Cloud & DevOps: AWS (EC2, S3, SageMaker), Linux System Administration, Bash Scripting. Leadership in Tech: As an active member of the Cloud Computing & DevOps Club (CCDC) since 2024, I have helped fostering a community of cloud enthusiasts and sharing hands-on knowledge of AWS services. Mission I am driven by the challenge of making "Efficient AI" accessible through superior cloud infrastructure. I am currently seeking opportunities to apply my skills in a research or professional intern- specifically focusing on cloud architecture and AI efficiency.. Whether it is optimizing a CI/CD pipeline or architecting a mobile app that utilizes the power of the cloud, my goal is to build technology that is secure, scalable, and impactful. My work often involves integrating advanced Machine Learning models—such as MobileNetV2—into mobile environments to solve real-world problems through computer vision and predictive analytics.

```


 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

```
# 1. First, tokenize to get the total count
all_sentences = sent_tokenize(text)
total_count = len(all_sentences)

# 2. Calculate 25% of the size
# We use max(1, ...) to ensure at least one sentence remains
target_count = max(1, int(total_count * 0.25))

# 3. Generate and print the summary
summary_25_percent = textrank_summarization(text, num_sentences=target_count)

print(f"Original sentences: {total_count}")
print(f"Summary sentences (25%): {target_count}")
print("\n--- Summary ---")
print(summary_50_percent)

...
Original sentences: 17
Summary sentences (25%): 4

--- Summary ---
My professional focus lies at the intersection of Cloud Infrastructure, Mobile Development, and Artificial Intelligence, where I build scalable, high-performance solutions that bridge the gap between complex backend systems and intuitive user experiences. Core Expertise
My technical foundation is built on a deep understanding of Cloud Engineering, specifically within the AWS (Amazon Web Services) ecosystem. Technical Toolkit
Cloud & DevOps: AWS (EC2, S3, SageMaker), Linux System Administration, Bash Scripting. Leadership in Tech: As an active member of the Cloud Computing & DevOps Club (CCDC) since 2024, I have helped fostering a community of cloud enthusiasts and sharing hands-on knowledge of AWS services.
```

```
text = """Mohammed Hamid | Cloud & Software Engineering Student
I am a final-year Software Engineering student at Marwadi University, Rajkot, currently in my sixth semester.
My professional focus lies at the intersection of Cloud Infrastructure, Mobile Development, and Artificial Intelligence,
where I build scalable, high-performance solutions that bridge the gap between complex backend systems and intuitive user experiences.

Core Expertise
I am a final-year Software Engineering student at Marwadi University, Rajkot, currently in my sixth semester.
My professional focus lies at the intersection of Cloud Infrastructure, Mobile Development, and Artificial Intelligence,
where I build scalable, high-performance solutions that bridge the gap between complex backend systems and intuitive user experiences.

As a developer, I am highly proficient in the Flutter framework, creating cross-platform mobile applications that are both robust and user-centric. My work often involves integrating advanced Machine Learning models—such as MobileNetV2—into mobile environments to solve real-world problems through computer vision and predictive analytics.
Key Projects & Impact
Agri-Tech AI Module: Currently developing a mobile-integrated AI solution using Flutter and AWS. This project leverages deep learning to diagnose plant diseases from leaf images, providing farmers with instant, actionable insights.
Leadership in Tech: As an active member of the Cloud Computing & DevOps Club (CCDC) since 2024, I have helped organize technical workshops, fostering a community of cloud enthusiasts and sharing hands-on knowledge of AWS services.
Certified Specialist: I hold the Cisco Linux Essentials certification and have completed the YUVA AI for All program via Simplilearn, demonstrating a verified commitment to continuous learning in networking and artificial intelligence.

Technical Toolkit
Cloud & DevOps: AWS (EC2, S3, SageMaker), Linux System Administration, Bash Scripting.

Mobile & Web: Flutter/Dart, Android Development, API Integration.
AI/ML: Computer Vision, Model Deployment (Google Colab, Cloud-to-Mobile).
Networking: Cisco Configuration, Router Security, Network Protocols.

Mission
I am driven by the challenge of making "Efficient AI" accessible through superior cloud infrastructure. Whether it is optimizing a CI/CD pipeline or architecting a mobile app that utilizes the power of the cloud, my goal is to build technology that is secure, scalable, and impactful. I am currently seeking opportunities to apply my skills in a research or professional internship capacity, specifically focusing on cloud architecture and AI efficiency.."""
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: TextRank for Document Summarization	
Experiment No: 10	Date: 28/03/2026	Enrolment No: 92301733059

Missing Sentence Recommendation: The sentence regarding the "**Agri-Tech AI Module**" was likely omitted or ranked lower by TextRank.

- **Why it should be there:** It is your most significant practical application of your combined skills (AWS + Flutter + AI).
- **Why TextRank missed it:** TextRank ranks sentences based on "centrality." Because the Agri-Tech sentence contains very specific domain words (like "leaf images" or "farmers") that don't appear in other sections, it has lower "similarity" to the rest of the document compared to general sentences about "Cloud Engineering".
- **Overall Summary Rating: 4/5**